동적 계획법

작은 입력의 답을 적어 두고 다음 계산에 다시 씁니다.

먼저 작은 예로 살펴봅니다

동적 계획법

작은 입력의 답을 적어 두고 다음 계산에 다시 씁니다.

예: 피보나치

0과 1을 적고, 앞의 두 칸을 더해 1,2,3,5를 차례로 적습니다.

6-1 피보나치 수

구할 것

앞의 두 수를 더해서 다음 수를 만듭니다.

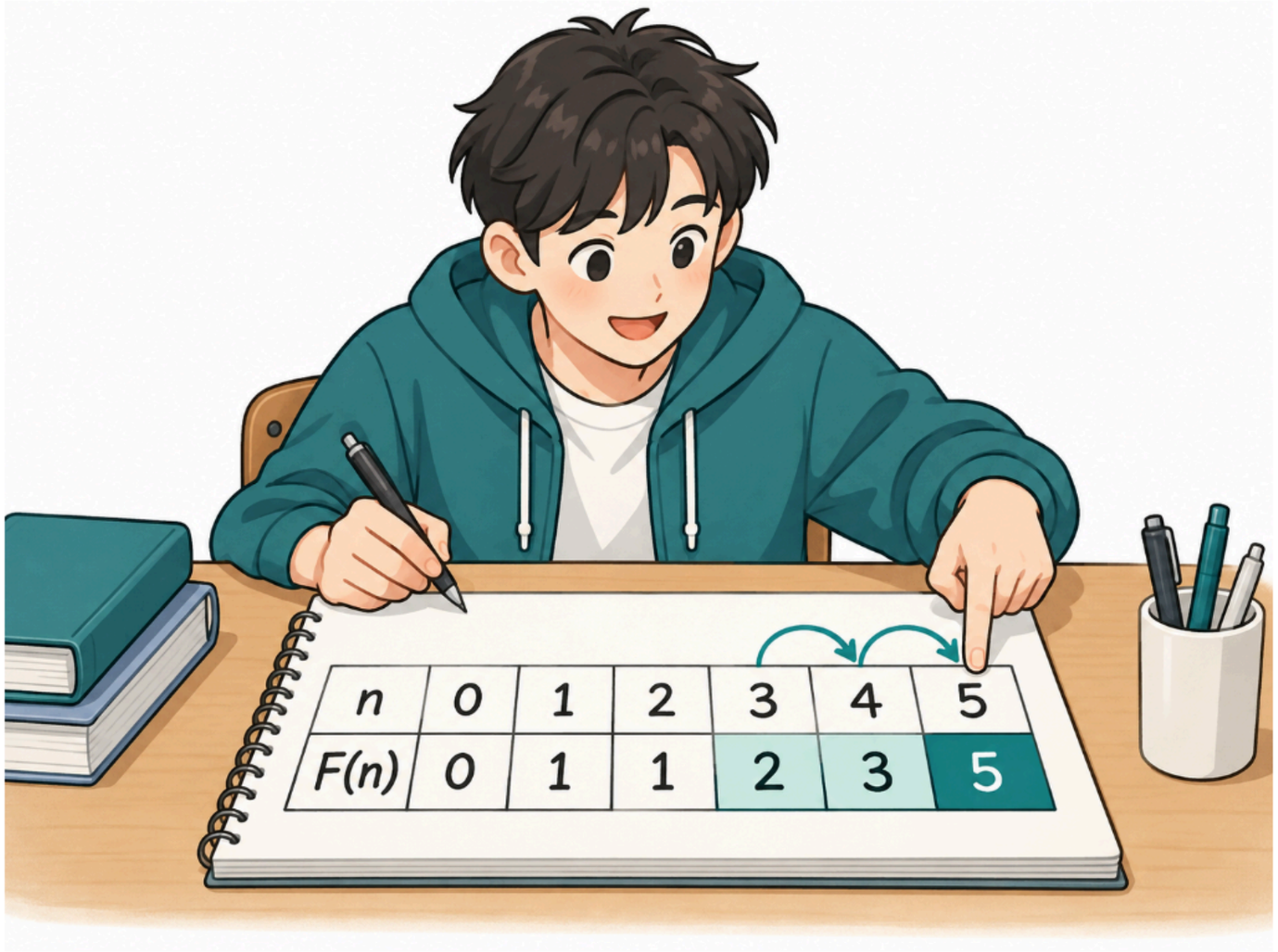
작은 예

처음 두 수는 0, 1입니다.
그다음은 1, 2, 3, 5, 8, 13입니다.

반환할 값

n=7이면 7번 칸의 값 13을 반환합니다. 번호는 0부터 셉니다.

6-1 그림으로 보는 예



0, 1, 1, 2, 3, 5
마지막 5는
앞의 2와 3을
더한 값입니다.

6-1 하나씩 계산한 결과

칸 번호 i	계산	적을 값
0	처음 값	0
1	처음 값	1
2	0 + 1	1
3	1 + 1	2
4	1 + 2	3
5	2 + 3	5

같은 방법으로 6번 칸은 8, 7번 칸은 13입니다.

6-1 계산 순서

저장할 것

dp[i]에는 i번째 피보나치 수를 적습니다.

처음 값

dp[0]=0, dp[1]=1입니다.

다음 값

dp[i]=dp[i−1]+dp[i−2]로 계산합니다.

6-1 변수와 코드의 뜻

코드	예: i=5일 때의 뜻
dp[i−1]	바로 앞의 값 dp[4]=3
dp[i−2]	두 칸 앞의 값 dp[3]=2
dp[i]	지금 구할 값 dp[5]=3+2=5
return dp[n]	요청한 번호의 값 하나를 반환

dp는 계산 결과를 적어 두는 배열의 이름입니다.

6-1 함수에 들어오는 값

함수

fib_bottomup(n)

입력 조건

n은 0~90인 정수입니다. C는 long long으로 반환합니다.

6-1 Python

```
def fib_bottomup(n):
    if n < 2:
        return n
    dp = [0] * (n+1)
    dp[1] = 1
    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

시간 O(n), 표 공간 O(n)

6-1 C

```
long long fib_bottomup(int n) {
    if (n < 2) return n;
    long long dp[91] = {0};
    dp[1] = 1;
    for (int i=2; i<=n; ++i)
        dp[i] = dp[i-1] + dp[i-2];
    return dp[n];
}
```

시간 O(n), 표 공간 O(n)

6-1 다른 입력도 확인합니다

입력	답	확인할 점
n=0	0	반복문을 돌 필요 없음
n=1	1	처음 값 그대로 반환
n=2	1	0+1 계산
n=7	13	번호와 값 구분

dp는 계산 결과를 적어 두는 배열의 이름입니다.

6-2 인접하지 않은 수의 최대 합

구할 것

붙어 있는 두 수를 함께 고르지 않고 합을 가장 크게 만듭니다.

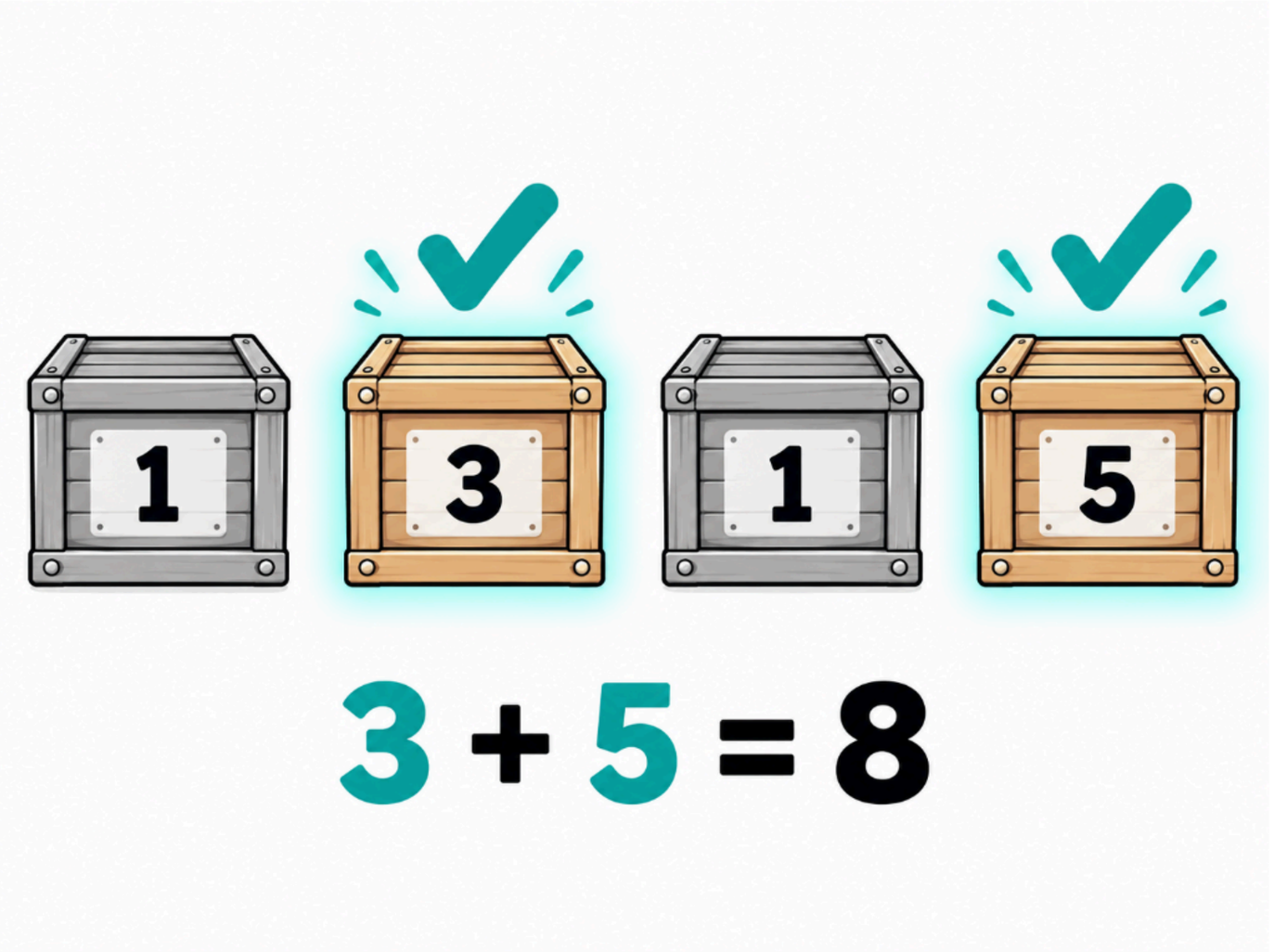
작은 예

배열이 [1, 3, 1, 5]라면 3과 5를 고를 수 있습니다.
두 수 사이에 1이 있으므로 서로 붙어 있지 않습니다.

반환할 값

가능한 최대 합 3+5=8을 반환합니다. 아무것도 고르지 않아도 됩니다.

6-2 그림으로 보는 예



[1, 3, 1, 5]
3과 5를 고릅니다.
둘은 붙어 있지
않습니다.
합은 8입니다.

6-2 하나씩 계산한 결과

이번 수	이번 수를 안 고르면	이번 수를 고르면	더 큰 값
1	0	0+1=1	1
3	1	0+3=3	3
1	3	1+1=2	3
5	3	3+5=8	8

마지막 5를 고를 때는 바로 앞의 1을 건너뜁니다.

6-2 계산 순서

안 고르는 경우

이전까지의 최대 합을 그대로 씁니다.

고르는 경우

바로 앞 원소를 빼고, 두 단계 전 최대 합에 이번 수를 더합니다.

이번 최대 합

두 경우 중 더 큰 값을 저장합니다.

6-2 변수와 코드의 뜻

변수	마지막 5를 처리하기 직전
value	이번 수 5
prev1	앞의 세 수 [1,3,1]에서 얻은 최대 합 3
prev2	앞의 두 수 [1,3]에서 얻은 최대 합 3
current	max(3, 3+5)=8

배열을 정렬하면 이웃 관계가 달라집니다. 주어진 순서로 계산합니다.

6-2 함수에 들어오는 값

함수

nonadjacent(a)

입력 조건

원소 0~10,000개. 각 값은 –1,000,000~1,000,000입니다. 빈 선택을 허용합니다.

6-2 Python

```
def nonadjacent(a):
    prev2 = 0
    prev1 = 0
    for value in a:
        skip = prev1
        take = prev2 + value
        current = max(skip, take)
        prev2 = prev1
        prev1 = current
    return prev1
```

시간 O(n), 추가 공간 O(1)

6-2 C

```
long long nonadjacent(const int a[], int n) {
    long long prev2=0, prev1=0;
    for (int i=0; i<n; ++i) {
        long long take=prev2+a[i];
        long long current=prev1>take ? prev1:take;
        prev2=prev1;
        prev1=current;
    }
    return prev1;
}
```

시간 O(n), 추가 공간 O(1)

6-2 다른 입력도 확인합니다

배열	답	이유
[]	0	고를 수가 없음
[-4,-2]	0	아무것도 안 고르는 편이 큼
[5,1,1,5]	10	양 끝의 5는 서로 붙어 있지 않음
[2,9,2]	9	2+2보다 9가 큼

배열을 정렬하면 이웃 관계가 달라집니다. 주어진 순서로 계산합니다.

6-3 1로 만들기

구할 것

정수 하나를 1로 바꾸는 데 필요한 최소 연산 횟수를 구합니다.

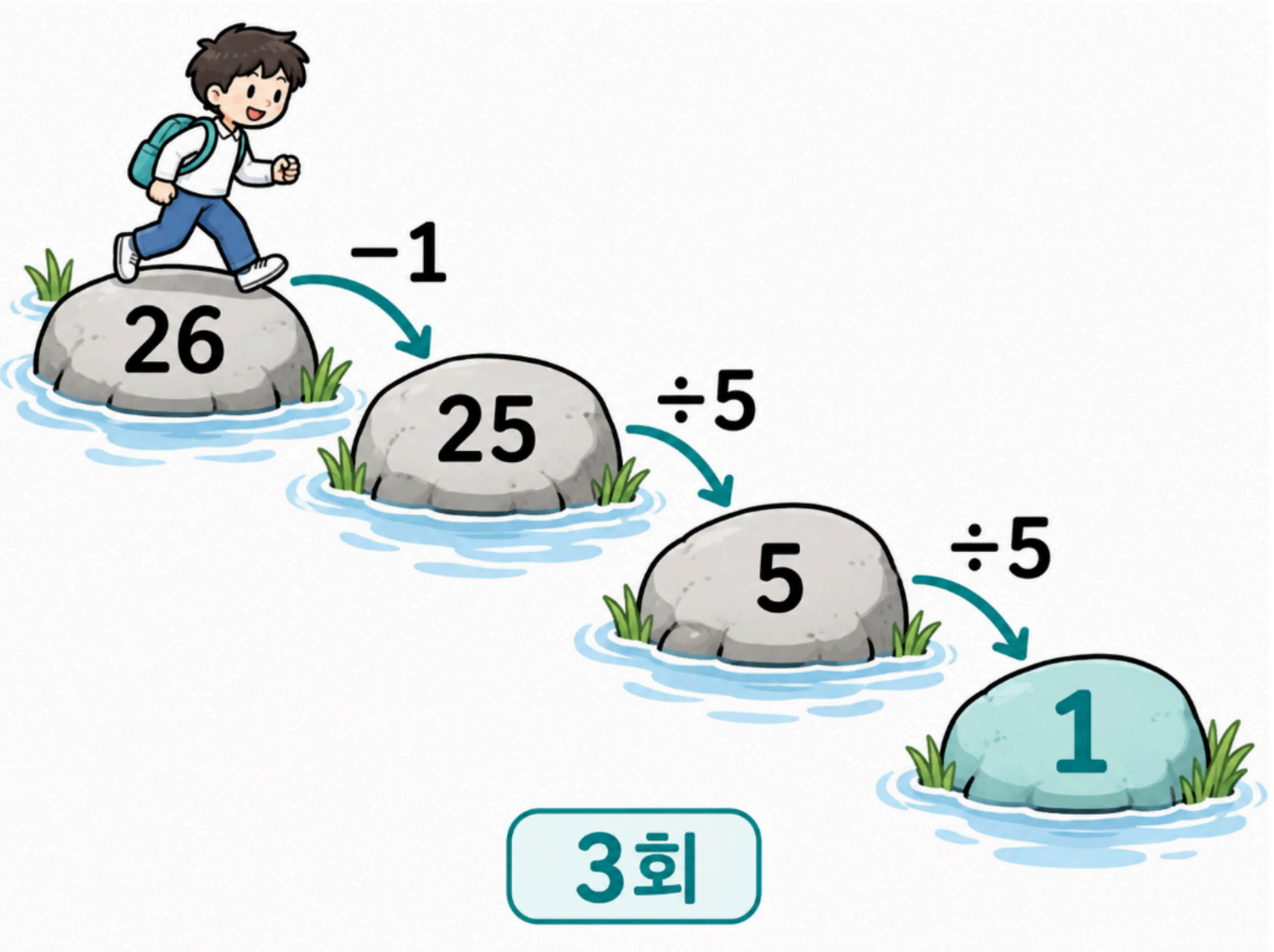
작은 예

한 번에 1을 빼거나, 딱 나누어떨어지면 2·3·5로 나눕니다.
10은 5를 거쳐 1로 가면 두 번입니다.

반환할 값

10÷2=5, 5÷5=1이므로 2를 반환합니다.

6-3 그림으로 보는 예



26에서 1을 빼면 25
25÷5=5
5÷5=1
총 세 번입니다.

6-3 하나씩 계산한 결과

현재 수 i	갈 수 있는 다음 수	각 후보의 횟수	최소
1	이미 1	0	0
2	1	1	1
3	2 또는 1	2 또는 1	1
4	3 또는 2	2 또는 2	2
5	4 또는 1	3 또는 1	1
6	5 또는 3 또는 2	2 또는 2 또는 2	2

다음 수까지 한 번 움직였으므로, 그 수의 답에 1을 더합니다.

6-3 계산 순서

저장할 것

dp[i]는 i를 1로 만드는 최소 횟수입니다.

먼저 계산할 후보

1을 뺀 i-1의 답에 1을 더합니다.

나누어떨어질 때

i÷2, i÷3, i÷5의 답에도 각각 1을 더해 가장 작은 값을 고릅니다.

6-3 변수와 코드의 뜻

코드	x=10에서의 계산
dp[x-1]+1	10에서 9로 이동: dp[9]+1=3
dp[x//2]+1	10에서 5로 이동: dp[5]+1=2
x % 3 == 0	10은 3으로 나누어떨어지지 않으므로 제외
dp[x//5]+1	10에서 2로 이동: dp[2]+1=2

큰 수부터 억지로 규칙을 정하지 않고, 작은 수의 답을 모두 계산해 둡니다.

6-3 함수에 들어오는 값

함수

make_one(n)

입력 조건

n은 1~10,000입니다. 나누기는 나누어떨어질 때만 허용합니다.

6-3 Python

```
def make_one(n):
    dp = [0]*(n+1)
    for x in range(2, n+1):
        dp[x] = dp[x-1]+1
        for d in (2,3,5):
            if x % d == 0:
                dp[x] = min(dp[x], dp[x//d]+1)
    return dp[n]
```

시간 O(n), 표 공간 O(n)

6-3 C

```
int make_one(int n, int dp[]) {
    int divisors[]={2,3,5};
    dp[1]=0;
    for (int x=2; x<=n; ++x) {
        dp[x]=dp[x-1]+1;
        for (int j=0; j<3; ++j) {
            int d=divisors[j];
            if (x%d==0 && dp[x/d]+1<dp[x]) dp[x]=dp[x/d]+1;
        }
    }
    return dp[n];
}
```

시간 O(n), 표 공간 O(n)

6-3 다른 입력도 확인합니다

입력	답	한 가지 최단 과정
1	0	이미 목표
5	1	5, 1
10	2	10, 5, 1
26	3	26, 25, 5, 1

큰 수부터 억지로 규칙을 정하지 않고, 작은 수의 답을 모두 계산해 둡니다.

6-4 0/1 배낭

구할 것

무게 제한을 지키면서 물건의 가치 합을 가장 크게 만듭니다.

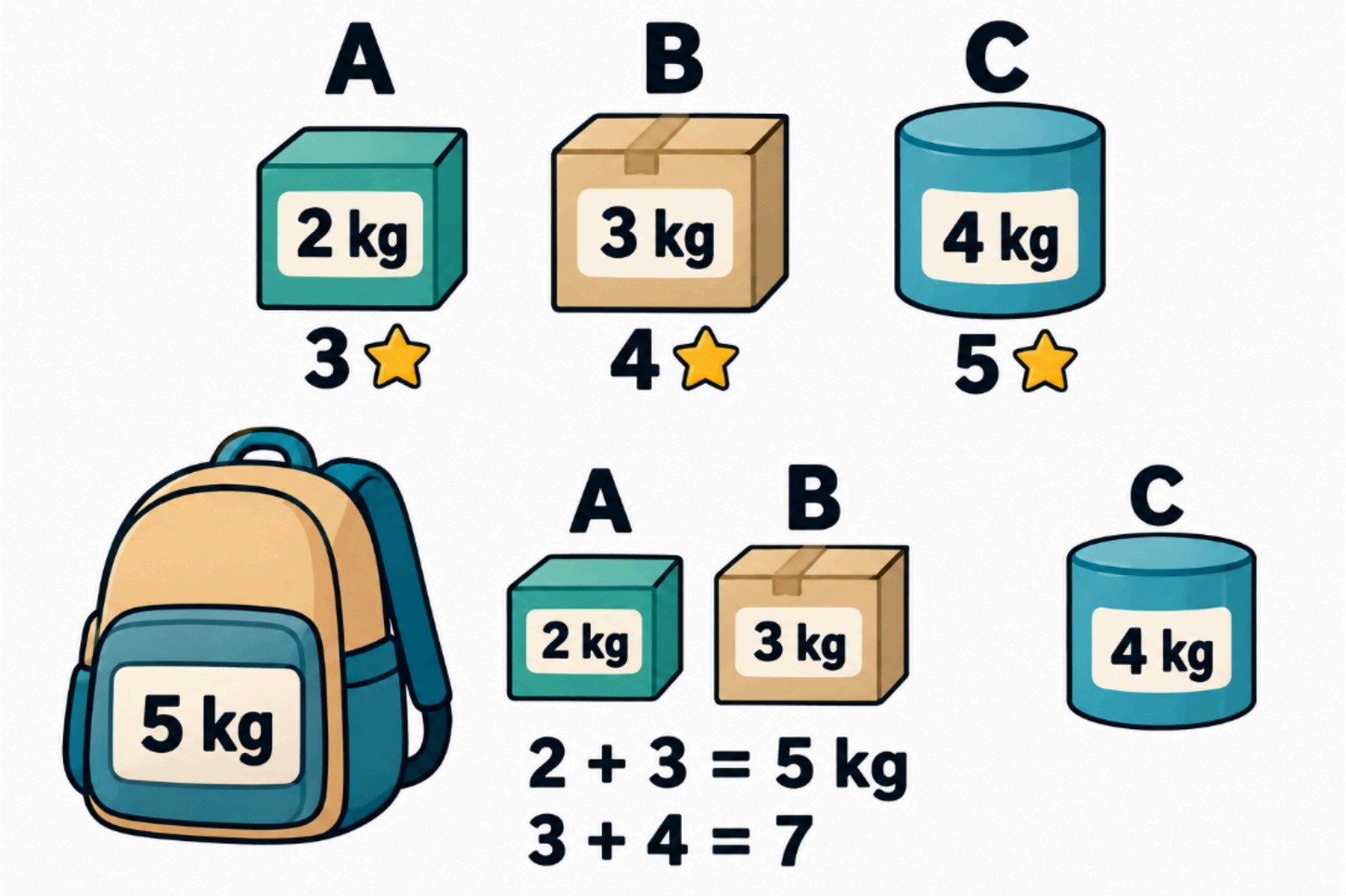
작은 예

배낭은 5kg까지 담습니다.
A는 2kg·3점, B는 3kg·4점, C는 4kg·5점입니다.

반환할 값

A와 B를 고르면 5kg·7점입니다. 최대 가치 7을 반환합니다.

6-4 그림으로 보는 예



배낭 용량은 5kg
A+B의 무게는 5kg
가치 합은 7점
각 물건은 한 번만
담을 수 있습니다.

6-4 하나씩 계산한 결과

사용할 물건	0kg	1kg	2kg	3kg	4kg	5kg
없음	0	0	0	0	0	0
A까지	0	0	3	3	3	3
A,B까지	0	0	3	4	4	7
A,B,C까지	0	0	3	4	5	7

표의 행은 사용할 수 있는 물건, 열은 배낭 용량입니다.

6-4 계산 순서

물건을 안 넣으면

바로 윗줄의 값을 가져옵니다.

물건을 넣으면

그 무게를 뺀 남은 용량의 윗줄 값에 이번 가치를 더합니다.

이번 칸

두 값 중 큰 값을 적습니다. 무게가 넘으면 넣는 경우는 제외합니다.

6-4 변수와 코드의 뜻

코드	B까지 보고 용량이 5kg인 칸
i=2, c=5	두 번째 물건 B와 용량 5
dp[i−1][c]	B를 안 담음: A만 쓸 때의 3점
dp[i−1][c−w]+v	B를 담음: 남은 2kg의 3점+4점
dp[i][c]	max(3,7)=7점

바로 윗줄을 읽으므로 같은 물건을 다시 담지 않습니다.

6-4 함수에 들어오는 값

함수

knapsack2(items, W)

입력 조건

물건 0~100개, 용량 0~1000. 무게 1~1000, 가치 0~1,000,000입니다.

6-4 Python

```
def knapsack2(items, W):
    n = len(items)
    dp = []
    for i in range(n+1):
        dp.append([0]*(W+1))
    for i in range(1, n+1):
        w, v = items[i-1]
        for c in range(W+1):
            dp[i][c] = dp[i-1][c]
            if w <= c:
                take = dp[i-1][c-w] + v
                dp[i][c] = max(dp[i][c], take)
    return dp[n][W]
```

물건 수 n, 용량 W: 시간·표 공간 O(nW)

6-4 C

```
int knapsack2(const int w[], const int v[], int n, int W,
    int dp[][1001]) {
    for (int c=0; c<=W; ++c) dp[0][c]=0;
    for (int i=1; i<=n; ++i) {
        for (int c=0; c<=W; ++c) {
            dp[i][c]=dp[i-1][c];
            if (w[i-1]<=c) {
                int take=dp[i-1][c-w[i-1]]+v[i-1];
                if (take>dp[i][c]) dp[i][c]=take;
            }
        }
    }
    return dp[n][W];
}
```

물건 수 n, 용량 W: 시간·표 공간 O(nW)

6-4 다른 입력도 확인합니다

입력	답	이유
물건 없음	0	담을 물건이 없음
용량 0kg	0	양의 무게 물건은 못 담음
2kg·3점 한 개, 용량 4kg	3	같은 물건을 두 번 사용하지 않음
A,B,C, 용량 5kg	7	A+B 선택

바로 윗줄을 읽으므로 같은 물건을 다시 담지 않습니다.

6장의 계산을 한 문장으로

문제	구하는 값
6-1	앞의 두 수를 더해서 다음 수를 만듭니다.
6-2	붙어 있는 두 수를 함께 고르지 않고 합을 가장 크게 만듭니다.
6-3	정수 하나를 1로 바꾸는 데 필요한 최소 연산 횟수를 구합니다.
6-4	무게 제한을 지키면서 물건의 가치 합을 가장 크게 만듭니다.

작은 입력의 답을 적어 두고 다음 계산에 다시 씁니다.